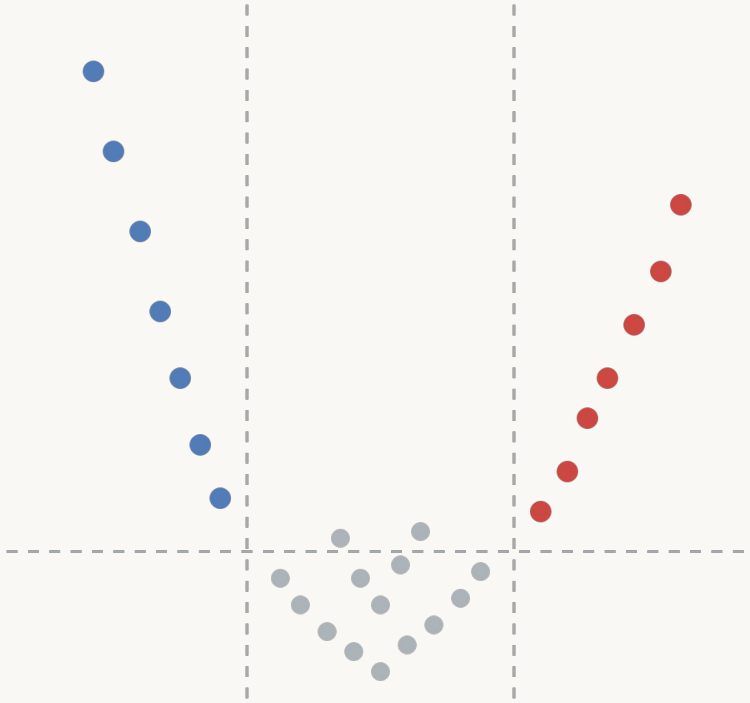
One regression per gene, thousands of tests



Module 2 · Differential expression with limma, and why we trust adjusted p-values

Francesco Cecere, Senior Bioinformatic Researcher
Eva Luna Tarallo, Junior Bioinformatic Researcher
AORN Santobono-Pausilipon IRCCS · BBCC2026 tutorial · TARGET-WT, 82 favorable, 42 unfavorable, 6 normal kidneys

Is this gene expressed differently in the tumour than in normal kidney?

Favorable vs control	82 favorable-histology tumours against 6 normal kidneys
Unfavorable vs control	42 anaplastic tumours against the same 6 normals

19,938 genes × 130 samples → filter → log2(TPM + 1) → a linear model per gene → p_adj

A test is only as good as its assumptions

limma’s p-values can be trusted when, for each gene, the log-expression behaves reasonably well: roughly symmetric noise, samples independent of each other, and enough signal to estimate how variable the gene is.

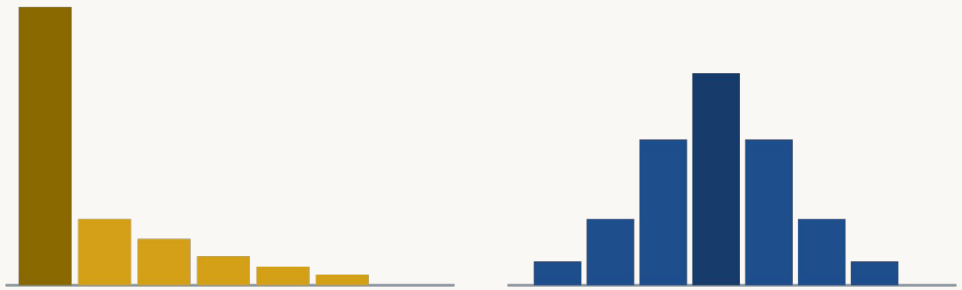
Barely expressed genes break these assumptions all at once.
That is why we filter before testing.

What the test assumes, and what breaks it

The test assumes	Broken by	We handle it with
effects add up	raw TPM, where effects multiply	the log scale
independent samples	a normal and a tumour from one child	duplicateCorrelation()
bell-shaped noise	low genes, full of zeros	filtering
a reliable variance	low genes, very noisy	filtering and limma’s shrinkage
genes that help each other	low genes distort the shared estimate	filtering and trend = TRUE

Three of the five fail mostly because of barely expressed genes. Removing them makes the test honest for the genes we keep.

At low expression the data are not bell-shaped



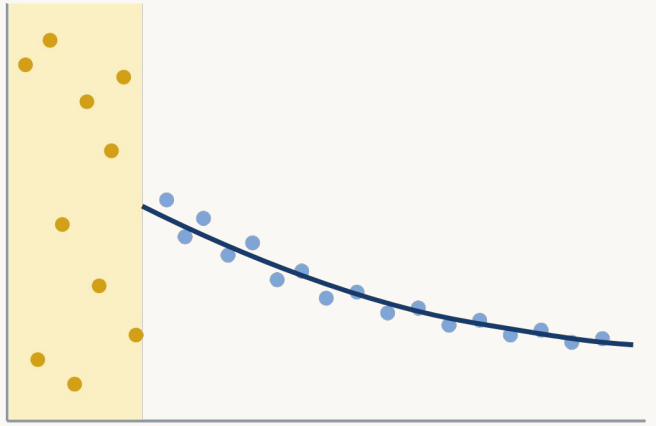
Schematic, one gene across samples. Left, a low gene piled up at 0. Right, an expressed gene, roughly bell-shaped.

Every non-detection lands on the same value, 0. A gene that is 0 in all 6 controls has no measurable spread in that group at all.

counting noise, as a share of the signal

reads for the gene	noise
1	100%
10	32%
100	10%
10,000	1%

For a barely detected gene the noise is as large as the signal.



Schematic.
x, how expressed a gene is.
y, how much it varies.
The shaded strip is what the filter removes.

Every useless test raises the bar for the useful ones

The more genes we test, the stricter the correction for multiple testing becomes, also for the genes that matter.

genes tested	the best gene needs p below
20,000	0.0000025
12,000	0.0000042

The filter must not look at the groups

We filter on overall expression, never on the difference between groups, otherwise the p-values would be biased.

```
keep_genes <- rowSums(tpm_matrix >= 1) >= n_control
log2_expr  <- log2(tpm_matrix[keep_genes, ] + 1)
```

TPM ≥ 1 in at least 6 samples, the smallest group, so a gene on only in the controls survives.

The regression limma applies

limma (Smyth, 2004) fits the same simple regression to every gene, then makes each gene's estimate of its own noise more reliable by borrowing information from all the others.

1

design

who is in which group

2

fit

lmFit(), one regression per gene

3

contrasts

the comparisons we ask

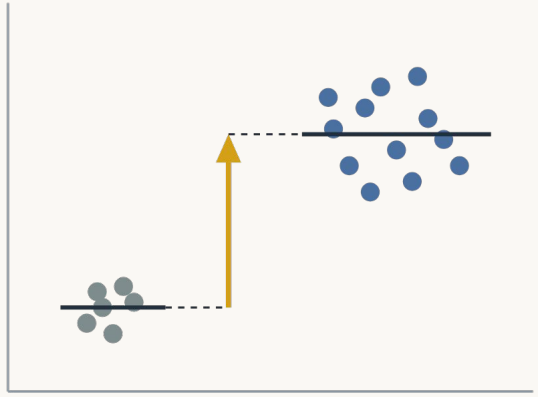
4

borrowing

eBayes(), the moderated t

One gene, one line

$$y = \beta_0 + \beta_1 \cdot x + \varepsilon$$



One gene. Grey, normal kidneys at x = 0.
Blue, tumours at x = 1.
The yellow step is β_1.

y	the gene's log2 expression in one sample
x	0 for a normal kidney, 1 for a tumour
β_0	the average level in normal kidney
β_1	how much the tumour differs, the log2 fold change
ε	patient-to-patient noise

The question "is this gene differentially expressed?" becomes "is β_1 different from 0?".

Three groups in one model

sample	Control	Fav.	Unfav.
favorable tumour	0	1	0
unfavorable tumour	0	0	1
normal kidney	1	0	0
samples	6	82	42

The design matrix, one row per sample, one column per group.

The same idea as the line, with one column per group.

limma estimates the average of each group for every gene.

```
design <- model.matrix(~ 0 + group,
                       data = de_samples)
fit    <- lmFit(log2_expr, design)
```

What a p-value is, and what it is not

If the gene did not change at all, how often would chance alone give a difference at least this large?

So among genes that truly do not change, 5% will still have $p < 0.05$.

It is not the probability that the gene is unchanged.

A p of 0.01 does not mean a 1% chance of no effect.

It is not the size of the effect.

A tiny change in a very precise gene can have a tiny p.

One test at 5% is fine.
Fifteen thousand are not.

If no gene changed at all and we tested 15,000 at $p < 0.05$, we would still find

$$\approx 750$$

"significant" genes, and every one of them would be false.

chance of at least one false positive

genes tested	at $p < 0.05$
1	5%
20	64%
100	99.4%
15,000	certain

Two ways to correct

Bonferroni

Protects against even one false positive in the whole list.

Very strict, so with thousands of genes almost nothing survives.

Suited to confirming one or a few genes.

False discovery rate

Controls the share of false calls in the list.

At 5%, a list of 100 genes contains about 5 false ones.

Suited to screening thousands of genes, so this is what we use.

We accept a few false positives in a long list, in exchange for many more true discoveries.

The adjusted p-value

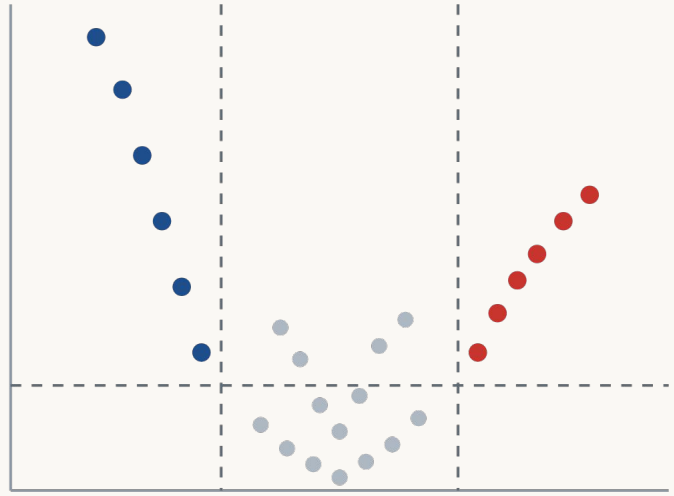
$$p_adj = p \times m / rank$$

p	the gene's raw p-value
m	how many genes we tested, the penalty for testing so many
rank	the gene's position when p-values are sorted, 1 for the smallest. The top gene is multiplied by m, the 100th only by m/100

This is the Benjamini–Hochberg correction.

Keeping genes with p_adj < 0.05 keeps the expected share of false discoveries at 5% or less.

A DEG needs evidence and a large enough effect



Schematic volcano. x, log2FC. y, −log10(p_adj). Grey points high in the centre are significant but small.

$$\text{DEG} \Leftrightarrow p_adj < 0.05 \text{ and } |log2FC| > 1$$

p_adj limits false discoveries, and |log2FC| > 1, at least doubled or halved, keeps only changes large enough to matter.

With a large n even tiny changes become significant. The effect size protects against that.

```
# the formal version tests |log2FC| > 1 itself
treat(fit, lfc = 1, trend = TRUE, robust = TRUE)
```